

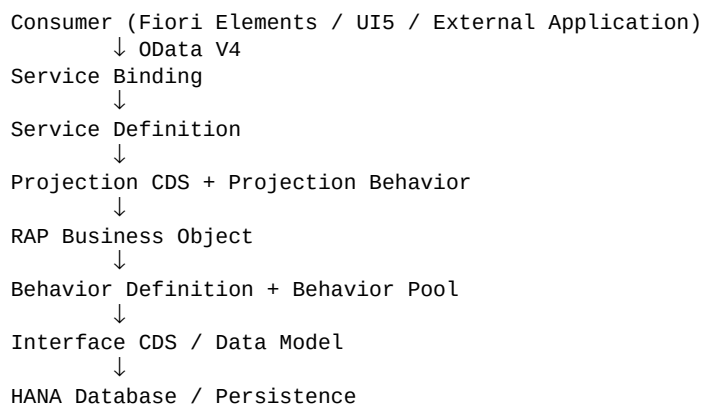
SAP RAP Application – Technical Architecture

Architecture, layers, request flow, managed vs unmanaged RAP, and interview perspective

1. Overview

SAP RAP (RESTful Application Programming Model) is a modern ABAP application programming model for building transactional, service-oriented applications on SAP S/4HANA and SAP BTP ABAP environments. It uses CDS-based data models, behavior definitions, ABAP behavior implementations, and OData services to expose business functionality.

2. High-Level Architecture



3. Main Architectural Layers

Persistence Layer – Database tables store the business data.

Data Model Layer – CDS view entities define business entities, fields, associations, and compositions.

Behavior Layer – Behavior definitions specify CRUD operations, actions, validations, determinations, locking, authorization, and draft capabilities.

Behavior Implementation – The behavior pool contains custom ABAP business logic.

Projection Layer – Projection CDS and projection behavior expose only the required part of the business object.

Service Layer – Service definition specifies which projection entities are exposed.

Service Binding – Publishes the service, commonly as OData V4.

UI/Consumer Layer – Fiori Elements, SAPUI5, or external applications consume the OData service.

4. CDS Data Model

The CDS layer defines the structure of the RAP business object. A typical transactional model contains a root entity and one or more child entities connected using associations and compositions.

Example:

Root Entity: Sales Order

■■■ Composition → Sales Items

The CDS model describes the business data and relationships, while transactional behavior is defined separately in the behavior layer.

5. Behavior Definition

The behavior definition (BDEF) declares what the business object can do. Typical capabilities include create, update, delete, actions, validations, determinations, field control, locking, authorization, and draft handling.

Example:

```
define behavior for ZI_SalesOrder
persistent table zsales_order
lock master
authorization master
{
create;
update;
delete;
action approve result [1] $self;
field ( readonly ) SalesOrder;
validation validate_customer on save;
determination set_status on modify;
}
```

6. Behavior Pool / ABAP Implementation

The behavior pool contains the ABAP implementation for custom transactional logic. For example, an action can update the status of a sales order using EML (Entity Manipulation Language).

The important separation is:

Behavior Definition → declares allowed behavior

Behavior Pool → implements custom behavior

7. Projection Layer

The projection layer separates the internal RAP business object from the externally exposed API. Projection CDS determines which entities and fields are exposed, while projection behavior controls which operations are available to consumers.

This separation improves API stability, security, reuse, and architectural decoupling.

8. Service Definition and Service Binding

The service definition identifies the projection entities that belong to the service. The service binding publishes that service to consumers, commonly using OData V4.

Flow:

Projection CDS → Service Definition → Service Binding → OData V4 → Consumer

9. End-to-End Request Flow

A typical Fiori request flows through the architecture as follows:

1. User performs an operation in Fiori Elements/UI5.
2. UI sends an OData V4 request.
3. Service Binding receives the request.
4. RAP service layer maps the request to the exposed projection.
5. RAP determines the relevant business object and behavior.
6. Authorization, locking, validations, determinations, and transactional processing are applied as appropriate.
7. Custom behavior logic executes in the behavior pool when required.
8. Persistence is handled by the RAP framework in a managed scenario or by custom implementation in an unmanaged scenario.
9. The response is returned through OData to the UI.

10. Managed vs Unmanaged RAP

Managed RAP – The RAP framework handles standard persistence and much of the transactional processing. The developer focuses primarily on business-specific behavior.

Unmanaged RAP – The developer controls more of the transactional and persistence implementation. This is useful when integrating existing legacy logic, BAPIs, function modules, or custom persistence mechanisms.

Conceptually:

Managed: Consumer → RAP Framework → Persistence

Unmanaged: Consumer → RAP Framework → Custom ABAP Logic → Existing API/Persistence

11. Important RAP Concepts for Architects

Business Object (BO) – The transactional business entity represented by root and child entities.

EML – ABAP syntax used to read and manipulate RAP business objects.

LUW / Transaction Handling – RAP coordinates transactional changes and save processing.

Draft – Supports editing scenarios where changes are temporarily stored before activation.

Authorization – Controls whether a user can access or change business data.

Feature Control – Controls whether fields, actions, or operations are enabled at runtime.

Validations – Check whether data is valid before save.

Determinations – Automatically derive or modify data based on business rules or events.

Actions – Business operations beyond standard CRUD.

12. Technical Architect Interview Answer

RAP follows a layered, metadata-driven architecture. The CDS data model defines the business object, the behavior definition defines its transactional capabilities, and the behavior pool implements custom business logic. A projection layer controls what is exposed externally. The service definition exposes the projection, while the service binding publishes it as an OData V4 service consumed by Fiori Elements, UI5, or other consumers. RAP provides framework-managed capabilities such as transactional processing, locking, authorization, validations, determinations, and draft handling. Depending on the scenario, persistence and CRUD processing can be managed by the RAP framework or implemented explicitly in an unmanaged scenario.

13. Architecture Summary

The key architectural chain to remember is:

